

3A : RISC-V ISA

Rajesh Panicker
ECE, NUS

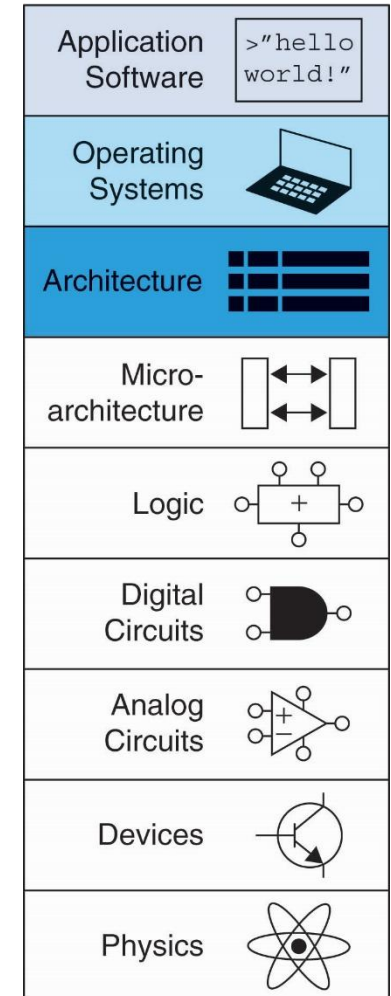
CG3207

Note:

- Not all slides will be covered in the lecture. The rest are left as a self-learning exercise
- Those in CAPS are ARM instructions / mnemonics for comparison
- Please ensure that you know ARM instruction set first (covered in EE2028/CG2028), as a lot of references and comparisons are made

Recap : Architecture vs Microarchitecture

- Architecture (ISA): programmer's view of computer
 - Defined by instructions & operand locations
 - Assembly language: human-readable format of instructions
 - Machine language: computer-readable format (1's and 0's)
 - Assembly language -> Machine language conversion is done by the assembler
 - one to one correspondence (except for pseudo-instructions)
- Microarchitecture: how to implement an architecture in hardware





Links and References

- Some tables are based on the work by (c) James Zhu. Modifications/corrections have been made to the original tables. The modified LaTeX files are available on request
- https://git.tu-berlin.de/j.koeppeler/riscv_tool/-/blob/master/riscvc_card_large.pdf - A fantastic reference card, which is almost all we need
- <https://raw.githubusercontent.com/mjosaarinen/rv-vkeccak-dev/refs/heads/main/riscv-spec.pdf> - RISC-V official specifications
- <https://en.wikipedia.org/wiki/RISC-V> - Wikipedia article on RISC-V. Has lots of info
- <https://riscv.org> - RISC-V website



RISC-V History

- Began in 2010 at the University of California, Berkeley as a short project
- Goal to make a practical ISA that was open-sourced, usable academically, and deployable in any hardware or software design without royalties
- Introduced in Aug 2014
- Transferred to the RISC-V Foundation in 2015, and then on to RISC-V International, a Swiss non-profit entity, in November 2019
- Fast gaining traction - numerous open source as well as commercial implementations
- Software support improving fast



Notable Products and Software

- ESP32-C3, ESP32-C6. ESP32-S series also have ultra-low-power cores which are RISC-V based though the main processors are Xtensa based
- Raspberry Pi Pico 2, Microchip FPGAs/SoCs
- Modern Nvidia GPUs and large processor chips include dozens of embedded NV-RISCV management cores
- Many implementations and commercial products - <https://en.wikipedia.org/wiki/RISC-V#Implementations>
- Many hardware accelerators built around RISC-V cores
- Mainline support for RISC-V ISA was added to the Linux kernel in 2022
- In July 2023, RISC-V, in its 64-bit variant called riscv64 was included as an official architecture by Debian



RISC-V Features

- As a RISC architecture, the RISC-V ISA is a load–store architecture – only load/store variants can access memory
 - No mixing of memory access with data processing or branching
- Interesting design choices to simplify hardware implementation
 - Especially the encoding of immediates
- Modular design - The instruction set is designed for a wide range of uses
 - The base instruction set has a fixed length of 32-bit naturally aligned instructions (different base variants such as RV32I, RV32E, RV64I,..exists)
 - ISA supports variable length extensions where each instruction can be any number of 16-bit parcels in length
 - Extensions include multiplication (M), floating point (F, D, Q), atomics (A), compressed instructions (C – 16-bit long instructions for high code density like ARM Thumb), ...

Note: Instruction lengths and word lengths are not necessarily the same



RISC-V Base Instructions

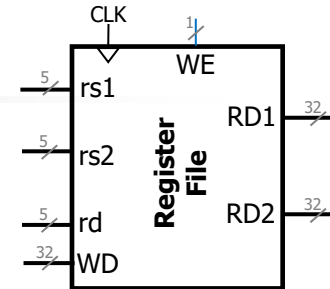
- Data Processing (DP): to process data
 - register: add, sub, or, xor, and, sll, srl, sra, slt, sltu
 - immediate: addi, ori, xori, andi, slli, srli, srai, slti, sltiu
 - upper immediate: lui (load upper immediate), auipc (add upper immediate to pc)
- Memory: to access memory
 - offset mode: lw, lh, lb, lhu, lbu, sw, sh, sb
 - No data processing. No pre/post index addressing
- Branch/Jump: to change control flow
 - Branch (conditional, no link): beq, bne, blt, bge, bltu, bgeu
 - Jump (unconditional, with link): jal, jalr
- Others: special purpose instructions
 - ecall (system call), ebREAK (breakpoint), fence (memory barrier)

Registers

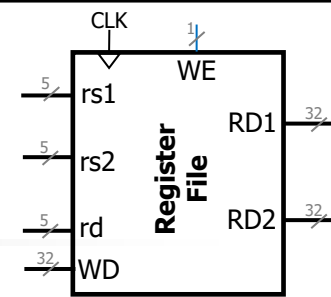
- There are 32 integer registers (x0-x31)
 - Unlike ARM which has 16 registers
- This means that each register needs a 5-bit identifier
 - This is 4-bit for ARM
- Description is the typical use (part of calling convention / ABI) for each register

Register	ABI Name	Description	Saver
x0	zero	Zero constant	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Fn args/return values	Caller
x12-x17	a2-a7	Fn args	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller

- There are also 32 floating-point extension registers if supporting floating point; as well as control and status registers (most not accessible in user mode)

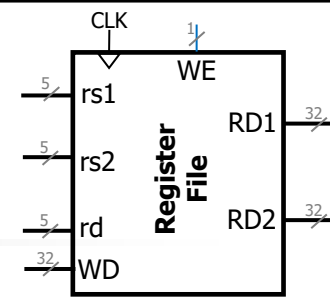


Registers



- Register x0 is hardwired to 0. Writes to x0 are ignored
 - Allows for easy comparison with 0
 - The equivalent of MOV is implemented using addition with x0, etc
- PC is not a register readable/writable explicitly by any instruction, i.e., it is not a visible register
 - Unlike ARM, where PC can be specified as an operand (R15)
- Reading from PC can be done indirectly through auipc, jal, and branch instructions, which uses PC itself as an operand
 - And not PC+8 like in ARMv3 or PC+4 like in ARMv7M / Thumb2
- Writing PC is done only by branch/jump instructions

Register Reads and Writes



- No instruction updates more than one visible register, and the register updated is explicitly specified in the rd field
 - This is unlike ARM where some instructions such as BL update a visible register (R14/LR) implicitly, i.e., without 14 explicitly specified in the instruction
 - This ensures that the register file needs only one write port
 - This has some implications such as not having pre/post-indexed addressing, multiply/division not giving 64-bit results, etc (discussed later)
- No instruction reads more than two registers. The register file needs only 2 read ports
 - This implies that instructions such as MLA (multiply and accumulate) are not possible



!Flag Registers

- There is no flag register(s) - flags are generated and used in the same instruction and not memorized for future use*
 - It was designed this way to minimize interaction between instructions
 - Only branch instructions are conditional, based on the result of a comparison done by the instruction itself - CMP + BEQ of ARM is replaced by beq
 - This also means the branch target computation cannot be done in ALU, as ALU does the comparison
- *If the result of a comparison is needed for future use other than branching, an instruction such as slt is used
 - Writes the result to a general-purpose register rather than to a dedicated flag register/flip-flop

Instruction Formats

This is different from ARM which has basically 3 formats : one for each category - DP (which has 3 sub-variants - DP Imm, DP reg shifted imm, DP reg shifter reg), Memory, and Branch.

- R (register : DP register)
- I (immediate : DP immediate, load - lw, jalr)
- S (store : sw)
- B, also referred to as SB (short branch : beq, bne, bge, blt, bgeu, bltu)
- U (upper: lui, auipc)
- J, also referred to as UJ (upper jump : jal)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

- The opcode field occupies the least significant part of the instruction, and RISC-V uses a little-endian format
 - This allows for the instruction type to be determined just by looking at the first byte alone (which occupies the lower memory address in little-endian)
- rs1, rs2, and rd bit positions standardized across formats
 - Saves some multiplexers

Instruction Formats : Immediate

Immediates in RISC-V assembly are not written with a # sign in front, unlike ARM.

- All immediates are MSB-extended - this implies sign-extension in almost all cases (but is a bit non-intuitive for sltu, where an unsigned number like 0xFFFFFFF8 is encoded as an immediate 0xFF8)
- Partial Verilog code for the Extend unit illustrates the commonalities that allow for hardware simplicity – saves muxes

```
ExtImm <= { Instr[31:20], Instr[19:12], 11'h000, 1'b0} ; // U
ExtImm <= {{12{Instr[31]}}, Instr[19:12], Instr[20], Instr[30:25], Instr[24:21], 1'b0} ; // J
ExtImm <= {{20{Instr[31]}}, Instr[31], Instr[30:25], Instr[24:21], Instr[20]} ; // I
ExtImm <= {{20{Instr[31]}}, Instr[31], Instr[30:25], Instr[11:8], Instr[7]} ; // S
ExtImm <= {{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0} ; // B
```

Instr Bits	ExtImm Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	U	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	0	0	0	0	0	0	0	0	0	0	0	0
	J	31	31	31	31	31	31	31	31	31	31	31	31	19	18	17	16	15	14	13	12	20	30	29	28	27	26	25	24	23	22	21	0
	I	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	30	29	28	27	26	25	24	23	22	21	20
	S	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	30	29	28	27	26	25	11	10	9	8	7
	B	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	7	30	29	28	27	26	25	11	10	9	8	0
	#muxes	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3	1	1	1	1	1	1	2	2	2	2	2

Those in **bold** are **0s** inserted by the Extend unit (as a part of left shift), not coming directly from Instr bits

DP Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Syntax
add	ADD	R	0110011	0x0	0x00	$rd = rs1 + rs2$	op rd, rs1, rs2
sub	SUB	R	0110011	0x0	0x20	$rd = rs1 - rs2$	
xor	XOR	R	0110011	0x4	0x00	$rd = rs1 \wedge rs2$	
or	OR	R	0110011	0x6	0x00	$rd = rs1 rs2$	
and	AND	R	0110011	0x7	0x00	$rd = rs1 \& rs2$	
sll	Shift Left Logical	R	0110011	0x1	0x00	$rd = rs1 \ll rs2$	
srl	Shift Right Logical	R	0110011	0x5	0x00	$rd = rs1 \gg rs2$	
sra	Shift Right Arith	R	0110011	0x5	0x20	$rd = rs1 \gg rs2$	
slt	Set Less Than	R	0110011	0x2	0x00	$rd = (rs1 < rs2)?1:0$	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	$rd = (rs1 < rs2)?1:0$	
addi	ADD Immediate	I	0010011	0x0	imm[11:5]=0x00	$rd = rs1 + imm$	op rd, rs1, imm
xori	XOR Immediate	I	0010011	0x4		$rd = rs1 \wedge imm$	
ori	OR Immediate	I	0010011	0x6		$rd = rs1 imm$	
andi	AND Immediate	I	0010011	0x7		$rd = rs1 \& imm$	
slli	Shift Left Logical Imm	I	0010011	0x1		$rd = rs1 \ll imm[4:0]$	
srli	Shift Right Logical Imm	I	0010011	0x5		$rd = rs1 \gg imm[4:0]$	
srai	Shift Right Arith Imm	I	0010011	0x5		$rd = rs1 \gg imm[4:0]$	
slti	Set Less Than Imm	I	0010011	0x2	imm[11:5]=0x20	$rd = (rs1 < imm)?1:0$	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		$rd = (rs1 < imm)?1:0$	
lui	Load Upper Imm	U	0110111			$rd = imm \ll 12$	lui rd, imm
auipc	Add Upper Imm to PC	U	0010111			$rd = PC + (imm \ll 12)$	auipc rd, imm

- Differences with ARM: EOR => xor, ORR => or, LSL => sll, LSR => srl, ASR => sra
- Shifts are real instructions, not variants of MOV unlike ARM. Rotate is not supported in RV32I
- Shift by an immediate makes use of imm[10] (same bit as funct7[5], which is Instr[30]) to distinguish between logical and arithmetic right shifts. This is ok since shift needs only 5 bits of imm

DP Instruction Example

- Arithmetic/logical instructions with immediates as the second operand have the suffix i (for example, add for register type, addi for immediate type)
 - Opcodes for add and addi are different too
- sub cannot take immediates
 - This is fine as the immediate is signed - we can simply use the negative of the value to be subtracted (known at assembly time) as the immediate for addi. $A-B = A+(-B)$
 - Also, funct7[5], which is used to distinguish between add and sub for register type is not available for immediate type

Pseudoinstruction/ Assembler Directive	Actual Instruction	Operation	Actual Memory Location Content (Instruction in Hex)
mv s2, s1	add x18, x0, x9	$x18 = x0 + x9 \Rightarrow x18 = x9$	0x00900933

31	27	26	25	24	20	19	15	14	12	11	7	6	0	R-type
funct7				rs2		rs1		funct3		rd		opcode		
0000 000				0 1001 (9)		0000 0 (0)		000		1001 0(18)		011 0011		

DP Pseudoinstruction - li

- la (load address) / li (load immediate) are pseudo instructions that are not 'load' in the strict sense of the word - no data memory access is involved
- 32-bit constants and absolute addresses (e.g., for MMIO) are generated using li which is implemented using lui and addi, without data memory access - 20 bits from lui and 12 bits from addi together form the 32 bits
- When li is used with small (12-bit) constants, it translates to addi alone; similar to MOV

Pseudoinstruction/ Assembler Directive	Actual Instruction	Operation	Actual Memory Location Content (Instruction in Hex)
li s1, 0x4321dcba	lui x9, 0x0004321e	x9 = 0x0004321e << 12 = 0x4321e000 [#]	0x4321e4b7
	addi x9, x9, 0xcba	x9 = x9 + 0xfffffcba [#] = 0x4321dcba	0xcba48493

- In contrast, in ARM, 32-bit addresses / constants are loaded from memory using the pseudoinstruction LDR Rx, =CONST_32/ADDRESS_32, which in turn is implemented as a PC-relative LDR and an assembler directive

[#] The 0xffff prefix to 0xcba is the result of MSB-extension. 0xffff is -1, which is why the immediate for lui is 0x0004321e to compensate

DP Pseudoinstruction - la

- 32-bit PC-relative addresses (la) are generated using auipc (facilitating position-independent code) and addi. 20 bits from auipc added to the most significant 20 bits of the PC, and 12 bits from addi make it 32-bits

Memory Address	Pseudoinstruction/ Assembler Directive	Actual Instruction	Operation	Actual Memory Location Content (Instruction in Hex)
0x00000010	la s1, LABEL	auipc x9, 2	$x9 = PC + imm \ll 12 = 0x00000010 + 2 \ll 12 = 0x00002010$	0x00002497
0x00000014		addi x9, x9, 0x3f4	$x9 = x9 + \text{MSB-extend}(0x3f4) \Rightarrow x9 = x9 + 0x000003f4$ (since imm is positive) = 0x00002404	0x3f448493
0x00002404	LABEL: .word 0xABCD1234	N.A.	N.A.	0xABCD1234

auipc	31	27	26	25	24	20	19	15	14	12	11	7	6	0	U-type
	imm[31:12]										rd		opcode		
	0000 0000 0000 0000 0010 (0x00002)										0100 1		001 0111		
addi	31	27	26	25	24	20	19	15	14	12	11	7	6	0	I-type
	imm[11:0]						rs1		funct3		rd		opcode		
	0011 1111 0100 (0x3f4)						0100 1 (9)		000		0100 1 (9)		001 0011		

Least Significant byte in
Lowest memory address in
Little-endian scheme
17

Memory Address	0x00000010	0x00000011	0x00000012	0x00000013	0x00000014	0x00000015	0x00000016	0x00000017
Actual Memory Location Content	0x97	0x24	0x00	0x00	0x93	0x84	0x44	0x3f

Multiply and Divide

- Multiply and Divide are not a part of the base instruction set, but are available as an optional standard extension (M)

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Syntax
mul	MUL	R	0110011	0x0	0x01	$rd = (rs1 * rs2)[31:0]$	op rd, rs1, rs2
mulh	MUL High	R	0110011	0x1	0x01	$rd = (rs1 * rs2)[63:32]$	
mulhsu	MUL High (S) (U)	R	0110011	0x2	0x01	$rd = (rs1 * rs2)[63:32]$	
mulhu	MUL High (U)	R	0110011	0x3	0x01	$rd = (rs1 * rs2)[63:32]$	
div	DIV	R	0110011	0x4	0x01	$rd = rs1 / rs2$	
divu	DIV (U)	R	0110011	0x5	0x01	$rd = rs1 / rs2$	
rem	Remainder	R	0110011	0x6	0x01	$rd = rs1 \% rs2$	
remu	Remainder (U)	R	0110011	0x7	0x01	$rd = rs1 \% rs2$	

- There are no instructions like SMULL, UMULL of ARM which update two registers, as only one register can be written by an instruction
 - To get a 64-bit result from multiplying two 32-bit numbers, we have to use mul and mulh/mulhu/mulhsu (depending on the signedness of the multiplicand and multiplier) for the least and most significant 32 bits respectively
 - Note that the signedness of the operand does not affect the least significant 32 bits of the multiplication result
 - mulhsu is a somewhat unique instruction not found in most other ISAs, allowing for a signed number to be multiplied with an unsigned number – useful for multi-word arithmetic

Memory Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Syntax
lb	Load Byte	I	0000011	0x0		$rd = M[rs1+imm][7:0]$	op rd, imm(rs1)
lh	Load Half	I	0000011	0x1		$rd = M[rs1+imm][15:0]$	
lw	Load Word	I	0000011	0x2		$rd = M[rs1+imm][31:0]$	
lbu	Load Byte (U)	I	0000011	0x4		$rd = M[rs1+imm][7:0]$	
lhu	Load Half (U)	I	0000011	0x5		$rd = M[rs1+imm][15:0]$	
sb	Store Byte	S	0100011	0x0		$M[rs1+imm][7:0] = rs2[7:0]$	op rs2, imm(rs1)
sh	Store Half	S	0100011	0x1		$M[rs1+imm][15:0] = rs2[15:0]$	
sw	Store Word	S	0100011	0x2		$M[rs1+imm][31:0] = rs2[31:0]$	

- lw and sw behave the same way as offset mode LDR/STR in ARM
- Note that lw and sw do not have the same format
- There are no pre/post-index modes - as only one register can be written by an instruction, base register and destination register can't be updated within the same instruction
 - Incrementing the base register has to be done explicitly using add/addi instructions
- No pre/post-index also means there are no load/store (LDM/STM) multiple type of instructions, and hence, no PUSH/POP (does this make the code slower? bigger?)
 - Implementing a stack (conventionally full descending) will need to be done entirely in software, using multiple separate lw/sw and addi instructions
 - Conventionally, sp - stack pointer, which is x2, is used

Memory Instructions - Pseudoinstructions

- Pseudoinstructions `lw rd, LABEL` and `sw rs2, LABEL, rt` (where `rt` is a temporary register) are also valid, which will be translated to
 - `auipc rd, imm1` `auipc rt, imm1`
 - `lw rd, imm2(rd)` `sw rs2, imm2(rt)`
 - where `imm1` and `imm2` are chosen by the assembler such that $\text{LABEL} = \text{PC} + \text{imm1} \ll 12 + \text{imm2}$
- For `lb` and `lh`, which loads a byte or a half word, the rest of the bits are formed by sign-extension of the byte/half-word
- For `lbu` and `lhu`, zero extension* is done instead
 - *Note that this does not contradict our earlier statement that RISC-V always MSB-extends the immediate. We are not talking about extending immediates here, but about extending the half-word or byte loaded from memory
 - In all cases, the offset used to generate the memory address is MSB-extended as it is an immediate
- `sb` and `sh` do not involve any extension, as only a byte or half-word in the memory is modified; the rest of the bytes/half-word within the word is unmodified

Memory Instruction Example

- Assuming $s1/x9 = 0x4321DCBA$; $s2/x18 = 0x00002408$

Pseudoinstruction/ Assembler Directive	Actual Instruction	Operation	Actual Memory Location Content (Instruction in Hex)
sw s1, -4(s2)	sw x9, 0xffc(x18)	$M[x18+0xffffffffc = 0x00002408+0xffffffffc = 0x00002404] = x9 = 0x4321DCBA$	0xfe992e23
lb s4, -4(s2)	lb x20, 0xffc(x18)	$x20[7:0] = M[x18+0xffffffffc = 0x00002408+0xffffffffc = 0x00002404]$ $x20[31:8] = x20[7]$ $x20 = 0xFFFFFFFFBA$ (0x000000BA if lbu)	0xffc90a03 (0xffc94a03 if lbu)

31	27	26	25	24	20	19	15	14	12	11	7	6	0		
imm[11:5]				rs2		rs1		funct3		imm[4:0]			opcode		S-type sw
1111 111 (0xffc)[11:5]				0 1001 (9)		1001 0 (18)		010		1110 0 (0xffc)[4:0]			010 0011		

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
imm[11:0]					rs1			funct3		rd		opcode		I-type
1111 1111 1100 (0xffc)					1001 0 (18)			000		1010 0 (20)		000 0011		lb

Control (Branch/Jump) Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Syntax
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	op rs1, rs2, LABEL*
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 ≥ rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 ≥ rs2) PC += imm	
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	jal rd, LABEL*
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	jalr rd, imm(rs1)

*Specify BTA as a LABEL and the assembler will encode the immediate.

- Itu (less than unsigned) = LO (lower); geu (greater or equal unsigned) = HS (higher or same)
 - Other conditions such as gt (GT), le (LE), gtu (HI), leu (LS) can be achieved by swapping the comparands as they can only be registers, not immediates
- Branches (conditional) and jal (unconditional) encode the offset of the target address (a PC-relative label) from PC (not PC+8 or PC+4)
 - RISC-V allows variable-length instructions but requires instruction lengths to be multiples of 16-bits, i.e., offsets are multiples of 2 -> LSB is 0 and is not encoded explicitly
 - The byte address offset is {imm[12:1], 1'b0} for branch, {imm[20:1], 1'b0} for jal

Control (Branch/Jump) Instructions

- jal can jump by a larger total\$ range (2^{21} bytes, i.e., 2^{20} half-words) from PC than is possible with conditional branches (2^{13} bytes, i.e., 2^{12} half-words). \$half of this to either side of PC
- There are two jump variants – jal and jalr
 - jalr (I format) is jump and link register, which is similar to BLX (branch target in a register) but allows for an offset from the base register
 - jal (I format) is jump and link, which is similar to unconditional BL (branch target is a PC-relative label)
- Unlike BL / BLX instructions, the register where the return address (link, i.e., PC+4) is stored has to be explicitly specified as rd (for both jal, jalr)
 - Conventionally, ra (x1) - return address is used (pseudoinstrn call)
 - If no return address needs to be stored, rd is set to x0. For example,
 - when returning from functions using jalr (pseudoinstruction ret)
 - when using jal for purposes other than calling a function (pseudoinstruction j J_BTA)
- Conditional branches (b*) do not allow for saving of return address

Branch/Jump Instruction Example

Memory Address	Pseudoinstruction/ Assembler Directive	Actual Instruction	Operation	Actual Memory Location Content (Instruction in Hex)
0x00000000	J_BTA: li s1, 1	addi x9, x0, 1	$x9 = x0 + 1 = 1$	0x00100493
0x00000004	SB_BTA: li s2, -2	addi x18, x0, 0xffe	$x18 = x0 + 0xfffffffffe = 0xfffffffffe$	0xffe00913
0x00000008	slt s4, s1, s2	slt x20, x9, x18	$x20 = x9 < x18?$ 1:0 (signed comparison) $x20 = 1 < -2?$ 1:0 = 0	0x0124aa33
0x0000000C	bne s4, zero, SB_BTA	bne x20, x0, 0xffffffff8 Note: BTA is specified as a LABEL. The byte offset, i.e., {imm[12:1], 0}, can be specified directly but is not usually done.	PC = (x20!=x0)? PC+0xffffffff8 : PC+4 PC = (0!=0)?(0x0000000C + 0xffffffff8) : (0x0000000C + 4) = 0x00000010 Branch won't be taken in this example	0xfe0a1ce3
0x00000010	jal J_BTA if rd is not specified, assembler will generate code for rd=ra / x1. j J_BTA sets rd=x0	jal x1, 0xffffffff0 Note: The note for the instruction above applies. The byte offset is {imm[20:1], 0}.	PC = PC+0xffffffff0 = (0x00000010 + 0xffffffff0) = 0x00000000 x1 = PC+4 = 0x00000014	0xff1ff0ef

31	27	26	25	24	20	19	15	14	12	11	7	6	0	B-type bne	
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]			opcode		
1 111 111 (0xfffff8[12 10:5])				0 0000 (0)		1010 0 (20)		001		1100 1 (0xfffff8[4:1 11])			110 0011		

31	27	26	25	24	20	19	15	14	12	11	7	6	0	J-type jal
imm[20 10:1 11 19:12]										rd		opcode		
1 111 1111 000 1 1111 1111 (0xfffff0[20 10:1 11 19:12])										00001 (1)		110 1111		



Summary

- RISC-V history; gaining traction fast
- DP, Memory, and Branch instruction features and encoding
- Coming next : Single-cycle microarchitecture

RV Assembly : Selection

```
// Doing static (storage class)
memory allocation here. The
variables are global to the
compilation unit /block/
function, and has a lifetime
that is the entire program)
```

```
static int x = x_init_val;
static int y = y_init_val;
static int z = z_init_val;
```

```
// inside some function
x++;
```

```
if(x>0)
{
    y++;
}
```

```
else
{
    y--;
}
```

```
z = x+y;
```

```
.data
# Declare vars x,y,z
x:  .word x_init_val
y:  .word y_init_val
z:  .word z_init_val
```

```
.text
# inside some function

# Load the values of 'x' and 'y' via lw pseudoinstrn
lw t0, x           # Load x into register t0
lw t1, y           # Load y into register t1

# x++
addi t0, t0, 1     # Increment x (t0 = t0 + 1)
sw t0, x, t3       # Store the updated value of x

# if (x > 0)
bgtz t0, greater_than_zero    # Branch if x > 0

# Else branch: y--
addi t1, t1, -1    # Decrement y (t1 = t1 - 1)
sw t1, y, t3       # Store the updated value of y.
                    # sw pseudoinstrn requires a temp register
                    # to be specified. Here, t3 is used throughout

    j update_z       # Jump to update_z to skip the 'then' branch

greater_than_zero:
    # y++
    addi t1, t1, 1   # Increment y (t1 = t1 + 1)
    sw t1, y, t3     # Store the updated value of y

update_z:
    # z = x + y
    add t2, t0, t1   # t2 = x + y
    sw t2, z, t3     # Store the value of z
```

RV Assembly : Iteration

```
// Doing static memory
allocation here.
// C variable / array
declarations not shown.
```

```
// inside some fn
for(i = 0; i < n; i++)
{
    x[i] = y[i] + 1;
}
```

```
.data
# Example array 'x' and 'y' with 5
elements each
```

```
x:  .word 0, 0, 0, 0, 0
y:  .word 1, 2, 3, 4, 5
```

```
# Number of elements, n = 5
n:  .word 5
```

```
.text
#inside some fn
lw t0, n # Load n into register t0 (t0 = n)
li t1, 0 # Initialize i to 0 (t1 = i)
la t4, y # Get the starting address of y
la t5, x # Get the starting address of x

loop:
    # Check if i < n
    bge t1, t0, exit_loop # If i >= n, exit loop

    # Load y[i] into t2
    slli t3, t1, 2        # i * 4 (for byte offset)
    add t6, t4, t3        # base + offset
    lw t2, 0(t6)         # Load y[i] into t2

    # x[i] = y[i] + 1
    addi t2, t2, 1        # t2 = y[i] + 1
    add t6, t5, t3        # base + offset
    sw t2, 0(t6)         # Store the result in x[i]

    # Increment i
    addi t1, t1, 1        # i++

    # Repeat the loop
    j loop

exit_loop:
    # Exit loop, to the rest of the function
```

RV Assembly : Function Call (Caller)

```
//Caller
void fn1() {
    //z1 and z2 are globals
    //x and y are local variables
    that use registers or stack
    int x = 20;
    int y = 30;
    z1 = fn2(x, y);
    x++;
    y++;
    z2 = x+y;
}

// Callee
int fn2(int p, int q) {
    int temp = p + q;
    return temp;
}

# Call fn2(20, 30), result in a0
mv a0, s0      # First argument (p = 20)
mv a1, t0      # Second argument (q = 30)
call fn2       # jal ra, fn2
sw a0, z1, t0   # Store the result (z1 = 50)

# Restore caller saved registers: a$, ra, t$
lw ra, 4(sp)   # Restore ra from stack
lw t1, 0(sp)   # Restore y (to t1 this time)
addi sp, sp, 8 # Restore sp

# x++ and y++
addi s0, s0, 1 # x++. x = 21
addi t1, t1, 1 # y++. y = 31
add t2, s0, t1  # x+y = 52
sw t2, z2, t3   # Store the result (z2 = 52)

# Epilogue: Restore callee saved regs: s$ modified by fn1
lw s0, 0(sp)   # Restore s0 from stack
addi sp, sp, 4 # Restore the stack pointer
ret           # Return to caller: jalr x0, ra, 0
```

RV Assembly : Function Call (Callee)

```
//Caller
void fn1() {
    //z1 and z2 are globals
    //x and y are local variables
    that use registers or stack
    int x = 20;
    int y = 30;
    z1 = fn2(x, y);
    x++;
    y++;
    z2 = x+y;
}
```

```
// Callee
int fn2(int p, int q) {
    int temp = p + q;
    return temp;
}
```

```
# Callee function.
# Arguments: p in a0, q in a1.
# Returns: temp (p + q) in a0
```

```
fn2:
    # Prologue: Save callee saved registers: s$ to stack
    # No need to save caller saved registers as fn2 does
    # not call any function (i.e., is not a caller)

    addi sp, sp, -4 # Make space on the stack
    sw s0, 0(sp)    # Save s0 to stack

    add s0, a0, a1   # temp = p + q
    mv a0, s0        # Return temp in a0

    # Epilogue: Restore callee saved registers: s$ from stack
    lw s0, 0(sp)     # Restore s0 from stack
    addi sp, sp, 4    # Restore sp

    jr ra           # Return to caller: jalr x0, ra, 0 (ret)
```



RV Assembly : Function Considerations

- For local variables, memory is allocated and deallocated automatically in the stack or are kept in register
- Modifying globals (z1, z2) from a function should be avoided as it goes against modularity of functions
 - Done here just for illustration
- There is no need to save and restore ra (caller saved) each time before calling a function
 - Sufficient to save it once at the beginning of a function and restore it at the end
- Read up more about RV calling convention – how data bigger than 32 bits are passed and returned, how a large number of parameters are dealt with, etc!
- Read up / experiment with how local variables are allocated in a function, stack frame, frame pointer, etc.
 - <https://godbolt.org/> is a great tool. Choose the correct language (C) and the compiler/ISA (RISC-V rv32gc clang)
- What if the callee function is farther than the range of jal?

Format for S and B – ISA Designer's Dilemma

- Store requires byte-level addressability to support sb, unaligned sw and sh. Hence, the least significant bit of the offset is relevant
- PC is always half-word aligned, so the least significant bit of the byte address offset is 0 and need not be represented
- Option 1: Same format for S and B
 - Simple in every sense, but lost opportunity to branch to farther instructions
- Option 2: RISC-V ISA Designer's Choice : different immediate formats – 'strange' looking for B
 - Incurs two 1-bit 2-1 muxes for bits 11 and 0, and possibly 1 extra control (ImmSrc) bit from the control unit. Actually, the mux for bit 0 is simply an AND gate (Why?). Total cost: A few gates extra
 - Benefit: Provides 2x the range of offset for B (i.e.; can branch to farther BTAs)

31	27	26	25	24	20	19	15	14	12	11	7	6	0		
imm[11:5]				rs2		rs1		funct3		imm[4:0]			opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]			opcode		B-type

ExtImm Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	30	29	28	27	26	25	11	10	9	8	7
B	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	7	30	29	28	27	26	25	11	10	9	8	0

Format for S and B – ISA Designer's Dilemma ...

- Option 3: A more 'regular' format where 12:1 for B occupies the bits occupied by 11:0 for S
 - Incurs 12 1-bit 2-1 muxes. Extra control (ImmSrc) bit from the control unit similar to option 2. Total cost: Many tens of gates extra
 - Machine code more readable, possibly slightly reduced assembler effort compared to option 2 (not really a practical consideration)
 - Benefit: Provides 2x the range of offset for B (i.e.; can branch to farther BTAs) similar to option 2

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12:6]				rs2		rs1		funct3		imm[5:1]		opcode		B-type

Instr Bits	ExtImm Bits		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
			31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S			31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	30	29	28	27	26	25	11	10	9	8	7
B			31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	31	30	29	28	27	26	25	11	10	9	8	7	0